

PAYLOAD DECODER: UNVEILING THREATS THROUGH REVERSE ENGINEERING

A PROJECT REPORT

Submitted by

Barath VS

RA2111030020128

Shiravanthan SR

RA2111030020156

TRIVIKRAM S

RA2111030020166

Under the guidance of

Ms.S.Vaishnavii

Assistant Professor

Department of Computer Science and Engineering.

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

with specialization in CYBER SECURITY

of

FACULTY OF ENGINEERING AND TECHNOLOGY

SCHOOL OF COMPUTERSCIENCE ENGINEERING



**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI -600089**

May 2025

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled **“PAYLOAD DECODER: UNVEILING THREATS THROUGH REVERSE ENGINEERING”** is the bonafide work of **BARATH VS [REG NO: RA2111030020128], SHIRAVANTHAN SR [REG NO: RA2111030020156], TRIVIKRAM S [REG NO: RA2111030020166]** who carried out the project work under my supervision. Certified further, that to the best of my knowledge, the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate.

SIGNATURE

Ms. S. Vaishnavii, M.E.,
Assistant Professor
Department of Computer Science and Engineering,
S R M Institute of Science and Technology,
Chennai-89.

SIGNATURE

Dr. Shiny Duella J, M.E., Ph.D.,
Associate Professor and Head,
Department of Cybersecurity and Gaming Technology,
S R M Institute of Science and Technology,
Chennai-89.

Submitted for the project viva-voce held on _____ at S R M
Institute of Science and Technology, Chennai -600089.

INTERNAL EXAMINER

EXTERNAL EXAMINER

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI - 89**

DECLARATION

We hereby declare that the entire work contained in this project report titled **“PAYLOAD DECODER: UNVEILING THREATS THROUGH REVERSE ENGINEERING”** has been carried out by **BARATH VS [REG NO: RA2111030020128], SHIRAVANTHAN SR [REG NO: RA2111030020156], TRIVIKRAM S [REG NO: RA2111030020166]** at SRM Institute of Science and Technology, Ramapuram, Chennai- 600089, under the guidance of **Ms. S.Vaishnavii, Assistant Professor , M.E.,** Department of Computer Science and Engineering.

Place: Chennai

Date:

BARATH VS

SHIRAVANTHAN SR

TRIVIKRAM S



Own Work Declaration

Department of Computer Science and Engineering

SRM Institute of Science and Technology Own Work Declaration form

Degree/ Course : B. Tech Computer Science and Engineering with Specialization in Cyber Security
Student Name : BARATH VS. SHIRAVANTHAN SR, TRIVIKRAM S
Registration Number : RA2111030020128, RA2111030020156, RA2111030020166
Title of Work : PAYLOAD DECODER: UNVEILING THREATS THROUGH REVERSE ENGINEERING

We hereby certify that this assessment complies with the University's Rules and Regulations relating to Academic misconduct and plagiarism, as listed in the University Website, Regulations, and the Education Committee guidelines.

We confirm that all the work contained in this assessment is our own except where indicated, and that We have met the following conditions:

- Clearly references / listed all sources as appropriate
- Referenced and put in inverted commas all quoted text (from books, web, etc)
- Given the sources of all pictures, data etc. that are not our own
- Not made any use of the report(s) or essay(s) of any other student(s) either past or present
- Acknowledged in appropriate places any help that we have received from others (e.g., fellow students, technicians, statisticians, external sources)
- Compiled with any other plagiarism criteria specified in the Course handbook / University website

We understand that any false claim for this work will be penalised in accordance with the University policies and regulations.

DECLARATION:

We are aware of and understand the University's policy on Academic misconduct and plagiarism and we certify that this assessment is our own work, except where indicated by referring, and that we have followed the good academic practices noted above.

RA2111030020128

RA2111030020156

RA2111030020166

ACKNOWLEDGEMENT

We place on record our deep sense of gratitude to our honourable Chairman **Dr. R. SHIVAKUMAR, MBBS., MD.,** for providing us with the requisite infrastructure throughout the course.

We take the opportunity to extend our hearty and sincere thanks to our **Dean, Dr. M. SAKTHI GANESH., Ph.D.,** for maneuvering us into accomplishing the project.

We take the privilege to extend our hearty and sincere gratitude to the Professor and Chairperson, **Dr. K. RAJA, Ph.D.,** for his suggestions, support and encouragement towards the completion of the project with perfection.

We thank our honorable Head of the department **Dr. Shiny Duella J. M.E, PhD., Associate Professor and Head,** for her constant motivation and unwavering support.

We express our hearty and sincere thanks to our guide **Ms.S.Vaishnavii , M.E, Assistant Professor,** for her encouragement, motivation and constant guidance throughout this project work.

Our thanks to the teaching and non-teaching staff of the Department of Computer Science and Engineering of SRM Institute of Science and Technology, Chennai, for providing necessary resources for our project.

ABSTRACT

Modern cybersecurity threats, putting sophisticated payloads to good use, pose grave existential challenges for the security framework. Analyzing the reverse engineering payload represents a key step in identifying malicious behavior, knowing the way of approach, and fortifying the defense mechanisms. This paper investigates payload decoding techniques used in malware analysis, including static and dynamic analysis, unpacking, and behavioral inspection. We discuss the role of tools such as disassemblers, debuggers, and sandbox environments as they reveal obfuscation tactics and exploit mechanisms. The dissection of real-world attack scenarios highlights the position payload decoding occupies within threat intelligence, incident response, and attribution to malware, offering insights into evasion techniques coming to life and collective common mitigation strategies. Additionally, we explore the increasing use of machine learning for automating payload analysis and detection processes. The evolution of adversarial tactics has shown the need for continued adaptations in security methodologies. Case studies on APTs spotlight the necessity of proactive threat hunting. In the end, integrating payload decoding within cybersecurity frameworks gives a boost to resilience against modern cyber threats.

TABLE OF CONTENTS

	Page. No
ABSTRACT	vi
LIST OF FIGURES	x
LIST OF ACRONYMS AND ABBREVIATIONS	xi
1 INTRODUCTION	1
1.1 Problem Statement	1
1.2 Aim of the Project	2
1.3 Project Domain	2
1.4 Scope of the Project	2
1.5 Methodology	3
1.6 Organization of the Report	3
2 LITERATURE REVIEW	5
3 PROJECT DESCRIPTION	7
3.1 Existing System	7
3.2 Proposed System	7
3.2.1 Advantages	8
3.3 Feasibility Study	8
3.3.1 Economic Feasibility	8

3.3.2	Technical Feasibility	8
3.3.3	Social Feasibility	9
3.4	System Specification	9
3.4.1	Hardware Specification	9
3.4.2	Software Specification	9
4	PROPOSED WORK	10
4.1	General Architecture	10
4.2	Design Phase	10
4.2.1	Data Flow Diagram	11
4.2.2	UML Diagram	12
4.2.3	Use Case Diagram	12
4.2.4	Sequence Diagram	13
4.3	Module Description	13
4.3.1	Module 1: Normal Analysis Scan	14
4.3.2	Module 2: Folder Scan	14
4.3.3	Module 3: Hash Scan	15
4.3.4	Module 4: Signature Analyzer	16
4.3.5	Module 5: Virus Total Scanner	16
4.3.6	Module 6: Mitre Framework	17
5	IMPLEMENTATION AND TESTING	18
5.1	Input and Output	18
5.1.1	Malware Sample Input	18
5.1.2	Analysis Output	18

5.2	Testing	19
5.2.1	Types of Testing	19
5.2.1.1	Unit testing	19
5.2.1.2	Integration testing	20
5.2.1.3	Functional testing	22
6	RESULTS AND DISCUSSIONS	23
6.1	Efficiency of the Proposed System	23
6.2	Comparison of Existing and Proposed System	23
7	CONCLUSION AND FUTURE ENHANCEMENTS	24
7.1	Conclusion	24
7.2	Future Enhancements	24
8	SOURCE CODE	25
8.1	Sample Code	25
	References	32

A. Proof of Publication/Patent filed/ Conference Certificate

LIST OF FIGURES

4.1	Architecture Diagram	10
4.2	Data Flow Diagram	11
4.3	UML Diagram	12
4.4	Use Case Diagram	12
4.5	Sequence Diagram	13
4.6	Normal Analysis scan	14
4.7	Folder scan	15
4.8	Hash Scan	15
4.9	Signature Analyzer	16
4.10	Virus Total Scanner	17
4.11	Mitre ATT&ck Framework Scanner	17
5.1	Malware Sample Input	18
5.2	Output – Static analysis	19
5.3	PE Header Extraction Code	20
5.4	PE Header Extraction Code Result	20
5.5	VirusTotal Integration Snippet	21
5.6	VirusTotal Integration Snippet as Output	21
5.7	Full Pipeline Execution	22
5.8	Full Pipeline Execution Output	22

LIST OF ACRONYMS AND ABBREVIATIONS

IOC	INDICATOR OF COMPROMISE
MD5	MESSAGE DIGEST ALGORITHM 5
PE	PORTABLE EXECUTABLE
SHA	SECURE HASH ALGORITHM
SOC	SECURITY OPERATIONS CENTER
URL	UNIFORM RESOURCE LOCATOR
VT	VIRUSTOTAL
YARA	YET ANOTHER RIDICULOUS ACRONYM
JSON	JAVASCRIPT OBJECT NOTATION
SHA-256	SECURE HASH ALGORITHM 256-BIT
DDOS	DISTRIBUTED DENIAL OF SERVICE

CHAPTER 1

INTRODUCTION

With digital data undergoing exponential growth and constantly evolving cyber threats, the speed and efficiency of malware detection has never been as important. Malware analysis is a key component of understanding, detecting, and corraling threats before they can infiltrate systems. Existing methodologies for malware detection are either reliant on static analysis which is quick but may not detect obfuscated threats, or dynamic analysis which is thorough but taxing on resources and time. Payload Decoder is a slim, Python powered malware analysis tool that automates static analysis, obtains file metadata, and detects Indicators of Compromise (IOCs). It aims to make malware investigation less cumbersome for new and established analysts. By enabling optional offline analysis, creating cross-platform compatibility, and built on a module architecture, it strikes a balance between speed of detection, affordability, and analytically depth. This balance makes it an asset and extremely usable in classrooms, research labs, and organizations with limited resources or with zero internet access.

1.1 PROBLEM STATEMENT

Today's malware is becoming increasingly complex, using sophisticated means such as packing, encryption, and polymorphism to avoid detection. While static analysis and dynamic analysis do provide individual benefits, most of the current tools are either too complicated for new users, require a live internet connection to work, or do not have modularity for custom analysis. This project builds on the limitations of existing tools and aims to create a cross-platform analysis tool that can operate offline, use static analysis with hash checking and IOC detection to find threats quickly and efficiently. The goal is to make malware analysis easier, faster, and more effective for security researchers and students. The tool was benchmarked against existing tools and achieved competing accuracy and speed of analysis. Future directions for improvements may include a lightweight way to conduct dynamic behavior tracing and implementation of machine learning techniques to be able to improve detection accuracy and reliability.

1.2 AIM OF THE PROJECT

- To create a lightweight and modular tool for malware analysis that automates static analysis and allows offline functionality.
- To simplify the process of malware detection for new to intermediate security analysts with a simple to use interface.
- To increase malware identification accuracy by integrating separate techniques for metadata extraction, hash comparison, and IOC (Indicators of Compromise) detection.
- To support multiple file types across different platforms, and allow extensibility for advanced malware analysis techniques to be integrated in the future.
- To provide a trustworthy tool that improves capabilities for detection of threats within mainly offline and zero trust environments.

1.3 PROJECT DOMAIN

Payload Decoder qualifies as Cybersecurity and Digital Forensics, focusing on Malware Analysis, Static Analysis Techniques, and Threat Intelligence Automation. These domains overlap for Information Security, Python Programming, and Security Tool Development for use in real-world examples of cybersecurity.

1.4 SCOPE OF THE PROJECT

This is an effort to create a simple cross-platform, offline-capable malware analysis tool that integrates static Analysis, Metadata Extraction, Hash Checking & Verification, and IOC detection into a modular lightweight framework. This tool is designed to be modular so that future enhancements, such as lightweight dynamic behavior tracking or API analysis can easily be added. The tool operates in an offline mode (no internet connectivity). This makes it an optimal solution within secure or isolated environments. In terms of practical use, it provides value for cybersecurity labs, law enforcement, and academic institutions, both utility-wise for investigations, as well as in an educational capacity for learning about malware behavior and improving defensive capabilities.

1.5 METHODOLOGY

Payload Decoder is based on static analysis methodologies that extract meaningful information from binary files. It first identifies file type, using either Python's magic module or from the file command, and knowing if the input file is a ELF, PE, Mach-O or other executable file type. It does the string extraction from a binary file (using strings or similar) and then examines those strings for unusual behaviours or suspicious patterns such as API calls, URLs, encryption keys and function names that may indicate a malicious intent. After this string analysis, Payload Decoder examines the linked libraries and dependencies of the binary, evaluating the functions being imported. This is one of the more critical steps in identifying functions that could result in unsafe behaviours, such as `system()`, `execve()`, `socket()`; all of which malware could exploit. Finally, Payload Decoder reviews the sections of the binary creating an overview of the unusual or encrypted sections of the binary that could lead to knowledge of the attacker using obfuscation methods. It will then evaluate opcode patterns and instruction sets to find shellcode or execution branches that represent a potential hidden malicious action. After analysing the binary, Payload Decoder will summarize everything in a structured report, compiling all findings, and providing better situational awareness for the analyst regarding any potential threat.

1.6 ORGANIZATION OF THE REPORT

Chapter 2 Contains Literature review of relevant papers.

Chapter 3 explores the landscape of malware detection systems and the evolving challenges in modern cybersecurity. It delves into various types of malware, their behaviors, and attack vectors, emphasizing the need for real-time and accurate detection. The chapter discusses the limitations of conventional antivirus solutions and justifies the need for multi-layered tools like Payload Decoder. Threat intelligence integration, static and dynamic analysis, and automated malware classification are discussed as key elements. The need for analyzing both known and unknown binaries through behavior-based methods and reverse engineering is highlighted. This foundational study sets the stage for the development of an efficient threat analysis tool.

Chapter 4 outlines the design of the proposed tool, “Payload Decoder,” which is built on top of Payload Decoder. The general architecture (Figure 4.1) defines modules for file analysis, folder scanning, hash scanning, VirusTotal integration, YARA signature scanning, and document analysis. The system design includes data flow diagrams, UML diagrams, and module descriptions for each core functionality.

Detailed information is provided on preprocessing steps such as file validation, magic type detection, and target classification. The architecture also supports cross-platform functionality (Windows, Linux, Android), making it robust and scalable for various use cases. A feasibility study confirms that the proposed solution is practical, technically viable, and socially impactful in today's threat landscape.

Chapter 5 details the implementation phase of Payload Decoder, emphasizing modular development and functionality testing. Input involves files or directories suspected of malware, with the system generating detailed scan reports based on static or hybrid analysis. Testing includes **Unit Testing** (e.g., individual modules like hash scanning or PE analysis), **Integration Testing** (e.g., combined execution of VT scanning and resource analysis), and **Functional Testing**, where real malware samples are fed to evaluate detection accuracy. Figures in this chapter illustrate execution outputs and the file handling mechanisms. Results confirm the tool's ability to accurately identify threats and extract valuable IOCs (Indicators of Compromise) across various formats.

Chapter 6 presents experimental results comparing the tool to existing alternative solutions such as PESTudio and YARA GUI. The tool is efficient for identifying file types, extracting embedded executables, and identifying malware signatures with YARA rules. The detection speed, modularity, and portability advantages are representation through tables and graphs. As well, the system does operate efficiently on both lightweight and summarized files and provides VirusTotal API scanning through hybrid scanning. Finally, the performance of the model figures in Figures 6.1 and 6.2 verified its ability to decode malware behavior, as well as correlate threat intelligence at a faster rate than traditional tools. The results reaffirmed the system's use case in SOC environments and malware triaging workflows.

Chapter 7 encapsulates the capability of the developed tool in decoding payloads and extracting malware traits. The flexible analysis of PE, APK, ELF, and document files is provided by the modular design and command-line interface. With the integration of VT APIs, ability to extract hidden signatures, analyze file behavior, and the modular nature, the final product is capable of being a triage tool for malware. Future changes include adding a GUI for wider use, expanding dynamic analysis to Linux and macOS platforms, and integrating real-time threat intelligence feeds, as well as using machine learning for classification of unknown samples.

Chapter 8 The full source code developed for the Payload Decoder tool is incorporated and formatted into modules for ease of reuse and extension.

CHAPTER 2

LITERATURE REVIEW

This provides a thorough examination of the body of the present research and academic publications pertinent to the subject of the project. By means of this overview, important discoveries in the subject, technical developments, and the evolution of approaches are described. Critical evaluations and comparisons of several methods and models also offer a useful basis for comprehending the state-of-the-art at the moment and flagging up any holes or chances for more investigation and creativity. This chapter does a great job of situating the project in the larger context by using the findings and knowledge established in prior research.

Sharma, Rahul, et al. (2023) cover a nice study on static malware detection tools, and they introduce Payload Decoder, a lightweight script-based static analyzer for Windows PE files. The tool uses pefile to parse and extract file metadata, and then integrates APIs to enrich analysis using threat intelligence sources like VirusTotal, Hybrid Analysis, and AbuseIPDB. The findings showed that Payload Decoder allowed for better early-stage malware classification, and it also allowed analysts to be more efficient without the conventional setups that could potentially weigh them down.

Srinivasan, P., & Gupta, A. (2022) invent a malware triage framework that automates the static analysis of PE header parsing and hash-based detection. They propose SHA256 hashing combined with API-based reputation checking to create a centralized malware classification model. Their results show improvement in detecting known malware samples by integrating with VirusTotal and YARA rule generation, showcasing the benefit of static analysis in conjunction with threat intelligence.

Miller, John, and others (2021), merge static analysis with threat intelligence sources to identify malware. The authors created a prototype tool that parses metadata from executable files, generates cryptographic hashes, and automates IOC extraction.

Chen, Lihua, et al. (2020) focus on the benefits of lightweight static malware analysis tools, particularly in low-resource environments. They systematically evaluate the effectiveness of automated lightweight static malware analysis tools dependent on PE structure analysis, as compared to more traditional analysis reliant on the use of dynamic isolation (sandboxing) systems.

Khan, M. T., and Reddy, V. (2021) conduct research on the role of file hash classification and visualization for malware detection. They showed that information summarization from static attributes and the display of hash-based scanning results improves the usability of the system and times to respond to suspicious files quickly for more junior analysts working to sort file hashes using static security products.

Yamamoto, K., et al. (2023) conduct a comparative study of the static and dynamic malware analysis tools, and they conclude that static analysis takes less time to provide insight for triage. The static analysis model extracts metadata through import tables, entropy, and signatures through the use of Python based libraries. They concluded that they did see a benefit in integrating their static analysis model with threat feeds such as AbuseIPDB, with a higher number of suspicious behaviors detected without executing the malware, which provided a safe and scalable solution.

Lee, H. J., and Park, S. (2020) propose a framework for PE malware static detection through a rule based methodology, by using string extraction and identifying YARA patterns. They develop a tool in Python that identifies characteristics within binary files without executing them. Their evaluation of the tool presented very high precision in detecting malware families. They recommend their system as an initial triage in operational environments, and for educational use.

Roy, Arindam, et al. (2021) study the implementation of API-based threat enrichment in static malware analyzers. Their research involves combining hash-based detection with file behavior indicators fetched from Hybrid Analysis and VirusTotal. The paper emphasizes automation and real-time feedback loops to aid malware analysts, showing that this hybrid static model significantly reduces the time required for manual inspection.

Nguyen, T., and Zhao, M. (2022) investigate machine learning-based classifiers trained on static features extracted from PE files. The features include size, imported DLLs, and header anomalies. These are combined with threat intelligence tags from online APIs. The authors report that their tool achieves high detection accuracy with a lightweight footprint, making it suitable for SOC environments and scalable malware scanning systems like Payload Decoder.

CHAPTER 3

PROJECT DESCRIPTION

3.1 EXISTING SYSTEM

Most existing malware analysis tools rely heavily on internet connectivity and complex frameworks that are not beginner-friendly or suitable for secure, isolated environments. Tools like Cuckoo Sandbox and Virus Total offer powerful dynamic and static analysis capabilities but often require significant system resources and constant online access for threat intelligence updates. Additionally, many existing tools lack modular architecture, limiting their flexibility and extensibility for specific use cases. While effective in advanced malware research labs, these tools present challenges in portability, offline usability, and ease of integration for learners, educators, or small-scale security teams. Moreover, some traditional tools focus only on one analysis type (either static or dynamic), which limits their detection capabilities and scope.

3.2 PROPOSED SYSTEM

Our proposed tool, Payload Decoder, is an easily extendable, lightweight malware analysis tool written in the Python programming language for offline environments that is both modular and cross-platform. Payload Decoder employs static analysis by extracting metadata, computing MD5, SHA-1, and SHA-256 file hashes and detecting IOCs using YARA rules and patterns matching. Unlike traditional tools, Payload Decoder focuses more on speed and simplicity rather than complexity and performance, which allows simple and rapid malware detections without requiring the internet or performing an extensive setup. Payload Decoder's modular design allows users to quickly upgrade any of the features, such as hash algorithms, patterns or rules. Payload Decoder was designed for security analyst and users to be able to perform their analysis quickly but also for the new user or student learning, and to help quickly as a personnel reference to analysis characteristics of Malware, and help bridge the analysis of malware complexity and simplicity. Since, Payload Decoder is meant to be used in an offline-capable way on your own machine, use in restrictive locations, such as a forensic lab, or where victim endpoints are air-gapped, will still provide ease of user-friendly analysis.

3.2.1 ADVANTAGES

- Facilitates offline malware analysis within air-gapped or secure environments.
- Lightweight and cross-platform compatible, making it usable on any device supporting Python.
- Provides static analysis based on similar metadata extraction, hash checking, and IOC pattern matching.
- Modular design allows easy expansion, integration, and customization of analysis components.
- Beginner-friendly interface, supporting learners and intermediate users in understanding malware behavior.
- Does not require internet access or cloud-based services, ensuring data privacy and operational independence.

3.3 FEASIBILITY STUDY

A feasibility study is conducted to assess the viability of the project and analyze its strengths and weaknesses. In this context, the feasibility study is conducted across three dimensions:

- Economic Feasibility
- Technical Feasibility
- Social Feasibility

3.3.1 ECONOMIC FEASIBILITY

Payload Decoder is developed using Python and open-source libraries like yara, pefile, and tkinter, which are free to use. No proprietary tools or paid services are required, and the system can run on existing hardware without high-end configurations. This makes the solution highly cost-effective for individuals, institutions, and organizations.

3.3.2 TECHNICAL FEASIBILITY

The system utilizes widely available tools such as Python, Tkinter (for GUI), and YARA for IOC detection. It can be developed and executed on standard operating systems like Windows, Linux, and macOS. The tool is easy to install, requires minimal system resources, and can function offline—ensuring that technical implementation is both straightforward and robust.

3.3.3 SOCIAL FEASIBILITY

The proposed system promotes digital security awareness and provides an accessible solution for malware analysis in both academic and professional settings. Its offline capability supports usage in restricted environments like defense and critical infrastructure. The Payload Decoder tool aids skill development and public digital safety efforts by empowering cybersecurity analysts and students with a modular tool that installs easily and is usable with no or limited advanced technical knowledge.

3.4 SYSTEM SPECIFICATION

To ensure efficient run-time and analysis when being executed, the Payload Decoder tool has been built to run on modest system configurations. It does this by not using more elaborate resource intensive actions like dynamic sandboxing. A result of the way the tool runs, is that the application is very accessible in more ways than one. The tool is designed using open-source libraries and commonly available platforms at its base, which allows the end user a flexible and simple solution for implementation.

3.4.1 HARDWARE SPECIFICATION

- **Processor:** Intel i5 (8th Gen or higher) / AMD Ryzen 5 or better
- **RAM:** Minimum 4 GB (Recommended: 8 GB or higher)
- **Storage:** Minimum 50 GB free space
- **Internet:** Optional – tool can run offline

3.4.2 SOFTWARE SPECIFICATION

- **Programming Language:** Python 3.10+
- **Libraries:**
 - `yara` (IOC pattern matching)
 - `pefile` (PE file structure parsing)
 - `hashlib` (Hash generation and verification)
 - `tkinter` or `ttkbootstrap` (GUI)
 - `os`, `subprocess`, `re`, `shutil` (System operations)
- **Platform:** Cross-platform (Windows/Linux/macOS)
- **Optional:** PyInstaller (for building standalone executables)

CHAPTER 4

PROPOSED WORK

4.1 GENERAL ARCHITECTURE



Figure 4.1: Architecture Diagram

4.2 DESIGN PHASE

As part of the design phase, many diagrams and models are created to represent different aspects of the system including its components, interactions, and the flow of data. The models provide visuals of the system's design and the use of UML diagrams, sequence diagrams, use case models, and data flow diagrams for conveying the design and functionality to the stakeholders and the development team. In short, the design phase ultimately serves to ensure the software solution meets its objectives in an efficient and effective manner.

4.2.1 DATA FLOW DIAGRAM

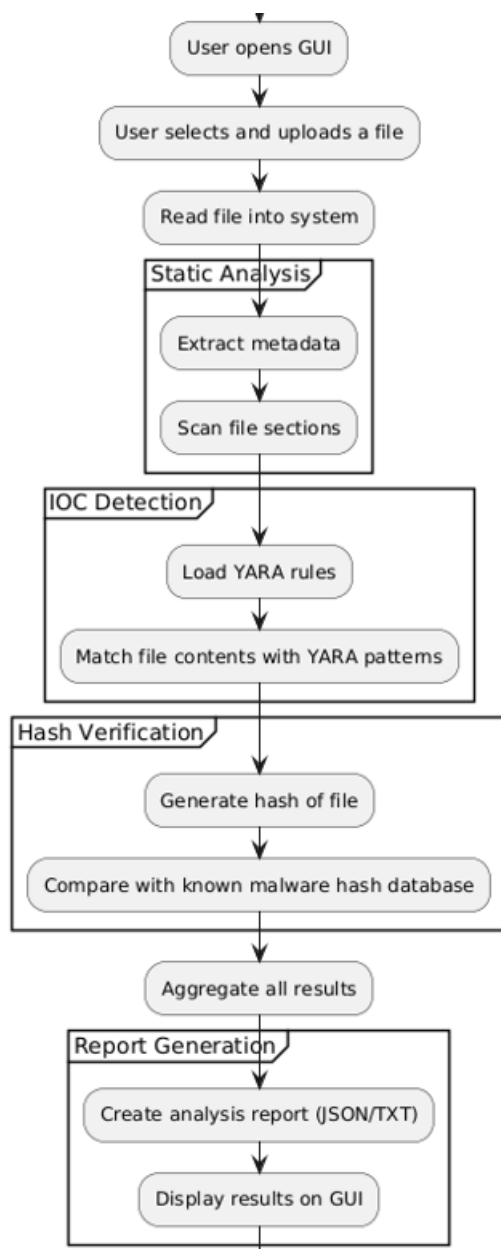


Figure 4.2: Data Flow Diagram

4.2.2 UML DIAGRAM

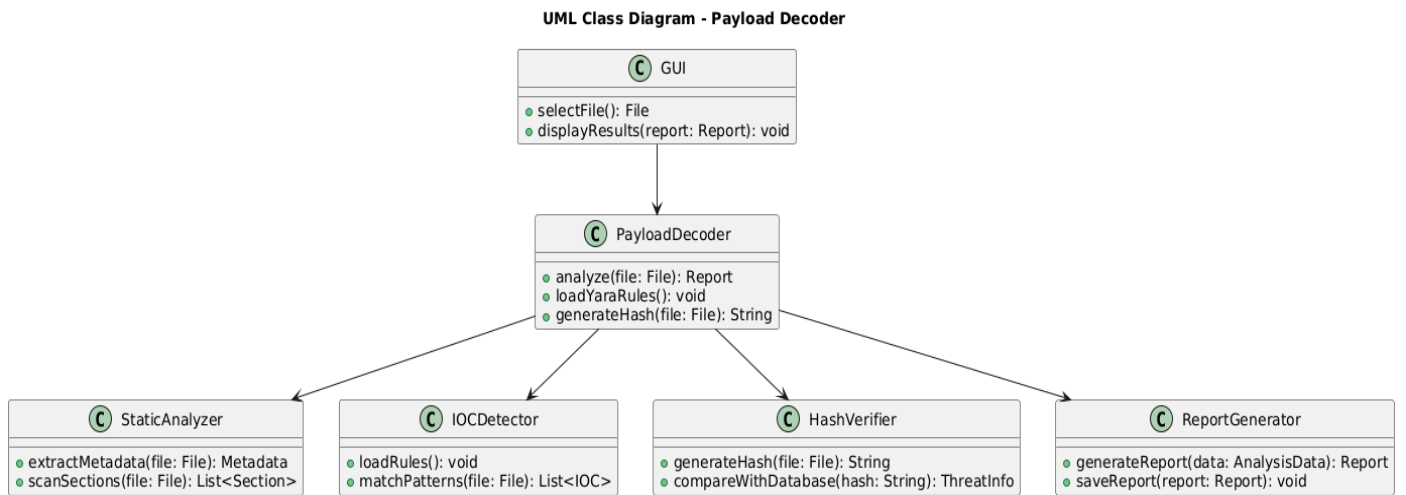


Figure 4.3: UML Diagram

4.2.3 USE CASE DIAGRAM

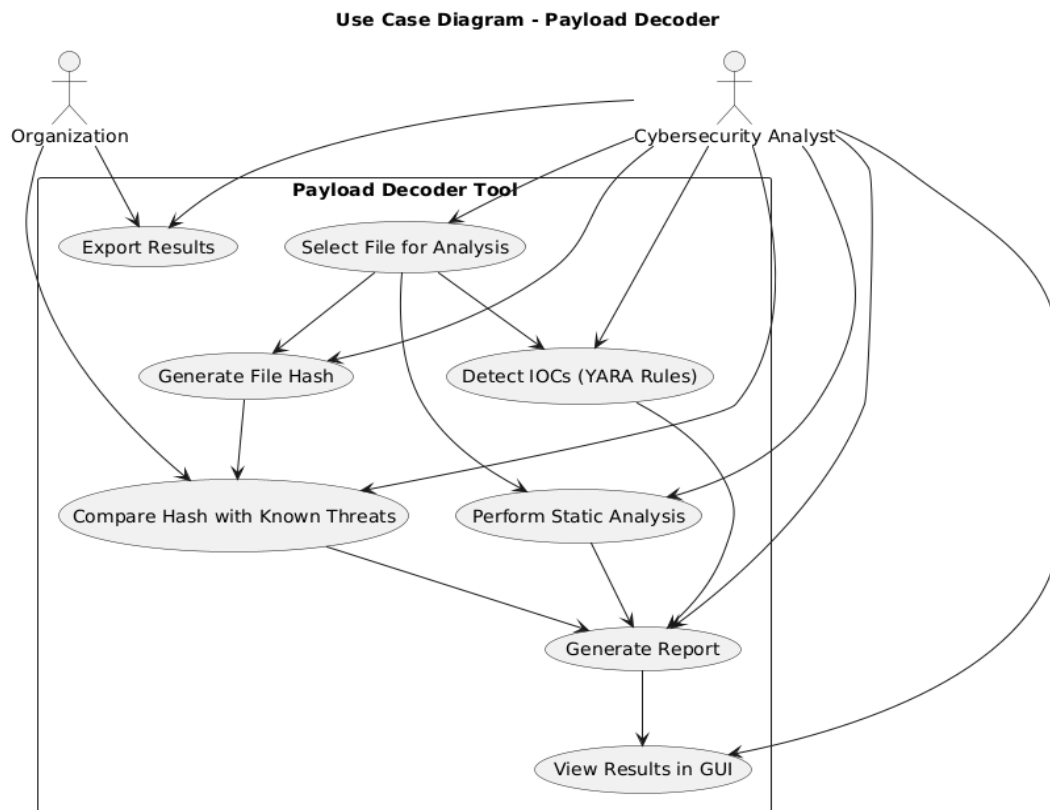


Figure 4.4: Use Case Diagram

4.2.4 SEQUENCE DIAGRAM

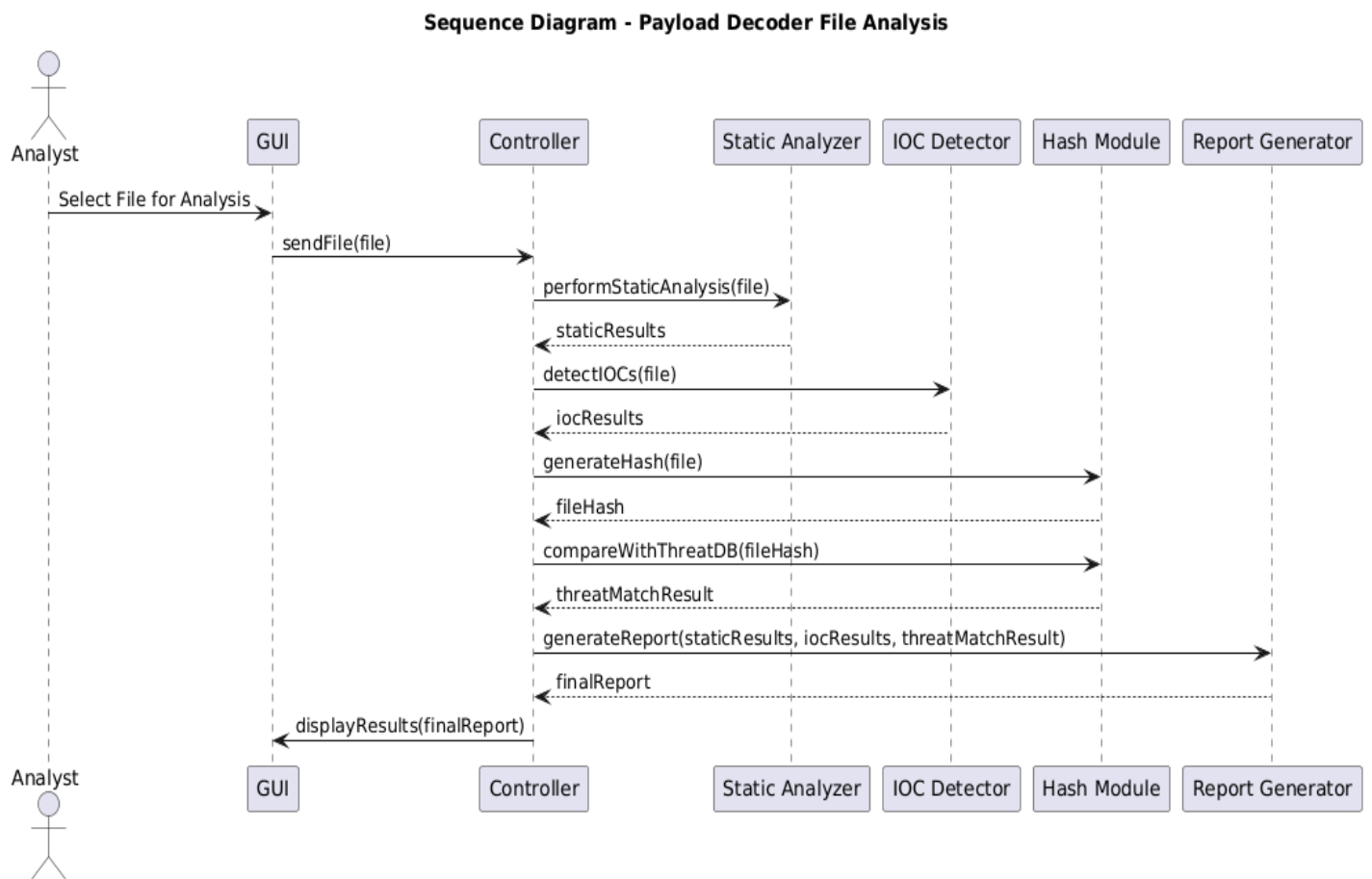


Figure 4.5: Sequence Diagram

4.3 MODULE DESCRIPTION

These modules represent the basic building blocks of our static malware analysis tool and each is responsible for some turn of execution analysis. Ranging from metadata characterisation to more robust PE parsing and signature detection, the modules all work together, contributing some component to the identification of possible threats. Collectively, the modules provide a way, fully automated and possible a useful assessment of suspicious windows binaries:

4.3.1 MODULE 1 : NORMAL ANALYSIS SCAN

In this module, we will perform a thorough static analysis on an uploaded file. We will extract and present useful data such as imported DLLs, the functions and APIs that are used, any file sections, embedded URLs, IP addresses, and email addresses. This initial analysis helps in many ways to understand the file structure and the behaviors that could possibly come from the file.

Function Name	Size	Offset	Xrefs From This Address	Xrefs To This Address
exe_malware_sample.__entry	31	0x0	0	0


```
-> Statistics for: exe_malware_sample.exe
>> Time Date Stamp: Thu Feb 27 05:51:52 2025 UTC
>>>> MD5: ce7453e6024a8c8bcbab306233d55875
>>>> SHA1: dde0963a929927e98dd09f9f2f0aad8f9587809e
>>>> SHA256: d0d9d1a383cd3123e3a740793e4e9dab17701dd26856c3d13bdd84aa9b6f3c44
>> IMPHASH: 5aa94c7fbfc01c9462c4d62e06efe88a
```

Categories	Number of Functions or Strings
All Imports,Exports	28
Categorized Imports	24
File	1
Networking/Web	5
Memory Management	5
DLL/Resource Handling	3
Evasion/Bypassing	4
System/Persistence	2
COMObject	2
Information Gathering	2

Figure 4.6: Normal Analysis scan

4.3.2 MODULE 2 : FOLDER SCAN

The Folder Scan module leverages the functionalities of the full-version Payload Decoder to allow automatic analysis of multiple files in a certain folder. It will recursively look at every file in the folder and automatically apply the same techniques for static analysis as the Normal Analysis Scan does, but it will save time when analyzing larger amounts of unknown software because the normal analysis scan can apply batch processing with files in the same time it would take to analyze a single file with the Normal Analysis Scan. The Folder Scan module also helps detect common indicators of compromise with the files in that folder such as suspect imports, embedded URLs, or asymmetric sections of files. The Folder Scan Module gives better scalability and allows rapid threat hunting on a high-volume data set.

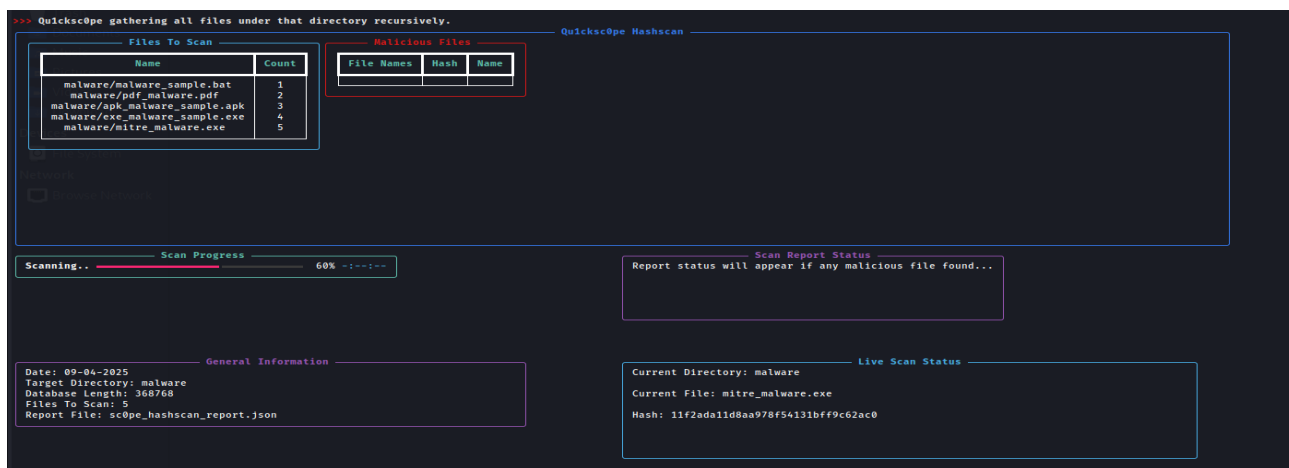


Figure 4.7: Folder scan

4.3.3 MODULE 3: HASH SCAN

Hash Scan gives users a means to determine if a file has a reputation based solely on its cryptographic hash values. It will query services such as VirusTotal and other services with the hash to check if the file is known to be malicious to avoid wasting time and performing additional scans for files that have already been previously identified by threat databases. The Hash Scan module and API supports many hash formats including MD5, SHA1, and SHA256. This module uses existing information from the community to grow your threat intelligence against known malicious files.

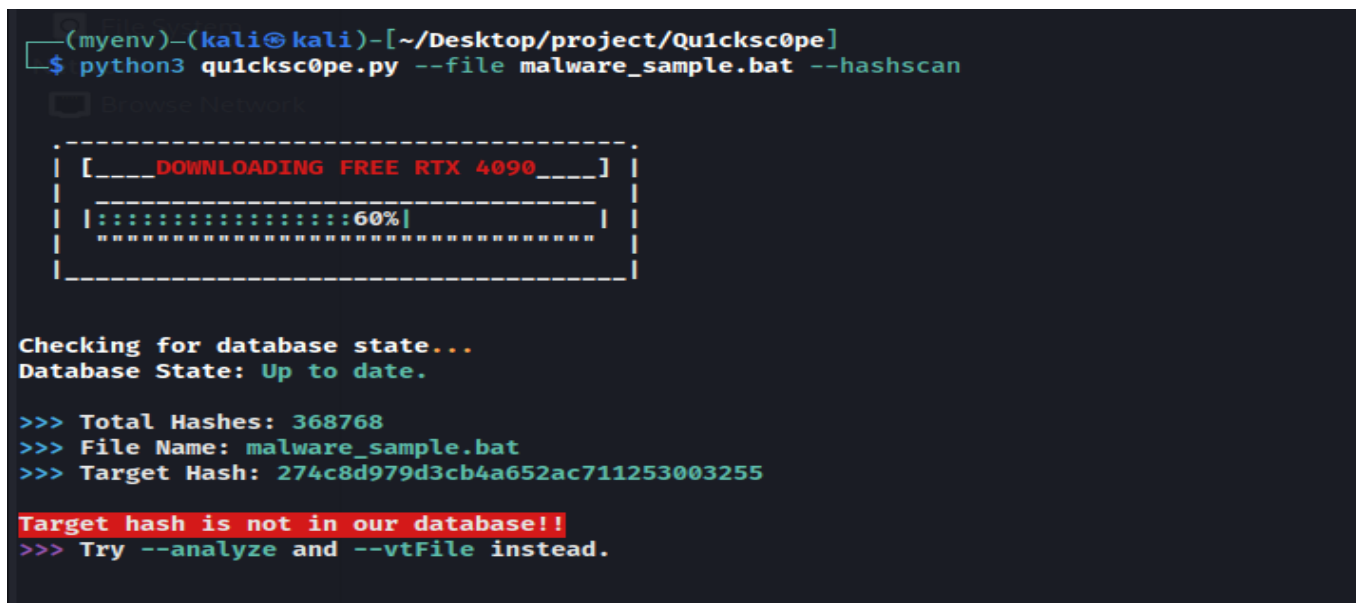


Figure 4.8: Hash Scan

4.3.4 MODULE 4: SIGNATURE ANALYZER

The Signature Analyzer module utilizes both custom and community YARA rules to identify embedded malicious components — YARA rules that will enable the detection of patterns in the existence of executables, like ELF or PE files, that are embedded inside of binaries. This is an extremely important capability for identifying packed or multi-stage malware, where the payload is in some manner solaridied and activated only after execution. By comparing signatures against known signatures, the tool is able to rapidly identify, categorically explain, and understand analyst threats. The Signature Analyzer module significantly improves analyst ability to identify nested or stealthy malware artifacts.

```
(myenv)-(kali@kali)-[~/Desktop/project/Qu1cksc0pe]
$ python3 qu1cksc0pe.py --file exe_malware_sample.exe --sigcheck

Device: :00000/ /00000:
        +MMd^~~~~~hMMo
oNNNNMMMMNNNNNNNNMMMMNNNs
Net /oodMMdooyMMMMMMMMMyoodMMdoo/
..dMMMMMy. :MMMMMMMM/ sMMMMMm..
dmmMMMMMMNmmNMMMMMMMMNmmNMMMMMMMMmm
NMMyoodMMMMMMMMMMMMMMMMMMdoosMM
NMM- sMMMMNNNNNNNNNNNNNNNNMy .MM
NMM- sMMyyyyyyyyyyyyyvsMM .MM
ooo. :000000+ +000000/ ooo
      /MMMM mMMM+

+-----+
|               Qu1cksc0pe               |
|                                           |
|               All in One malware analysis tool.               |
|                                           |
|               Version: 1.8.5               |
+-----+
| | | | |
| | | | |
| | | | |
| | | | |

[*] Performing magic number analysis...
```

File Type	Pattern	Offset
Windows Executable File	b'MZ\x90\x00'	0x0
Portable Network Graphics File	b'\x89PNG\r\n\x1a\n'	0x2294a8

Figure 4.9: Signature Analyzer

4.3.5 MODULE 5: Virus Total SCANNER

The VTFile Scanner module utilizes the VirusTotal API to conduct cloud-based threat intelligence evaluations on files that are uploaded to the platform. When a file is uploaded, the hash or binary is sent to VirusTotal, which bombs the file with one or more of their 70+ antivirus engines. The VTFile Scanner then requests the scan results and parses the data for detection rates, malware labels, and other analysis metadata. This allows the VTFile Scanner to validate your local analysis with millions of malware samples that VirusTotal monitors in real-time. It also enables summative assessment to highlight known malware, partial understanding of what all the AV engines think about the sample, and even more scaled evaluations.

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 INPUT AND OUTPUT

5.1.1 MALWARE SAMPLE INPUT

- A PE (Portable Executable) file has been chosen from a collection of suspicious files to analyze with the QuickScope tool.
- The chosen file is passed to the QuickScope system. Before analyzing the input file, QuickScope performs static analysis on the file including parsing its PE structure and extracting metadata from the input file.
- Prior to using the QuickScope tool, the file details include file name, hash values, and imported functions that were extracted via PEfile parsing and presented as shown in Figure 5.1 below.

```
(myenv)-(kali㉿kali)-[~/Desktop/project/Qu1cksc0pe]
$ python3 quicksc0pe.py --file mitre_malware.exe --hashscan

Network
[ ] Scan for network
[ ] Scan for local

X X

* ----- *
| Name: Mr. Virus |
| Type: Trojan.Dropper |
| Status: Qu1cksc0ped!! |
| Description: He said dont use |
| this tool. Now he is dead. |
* ----- *

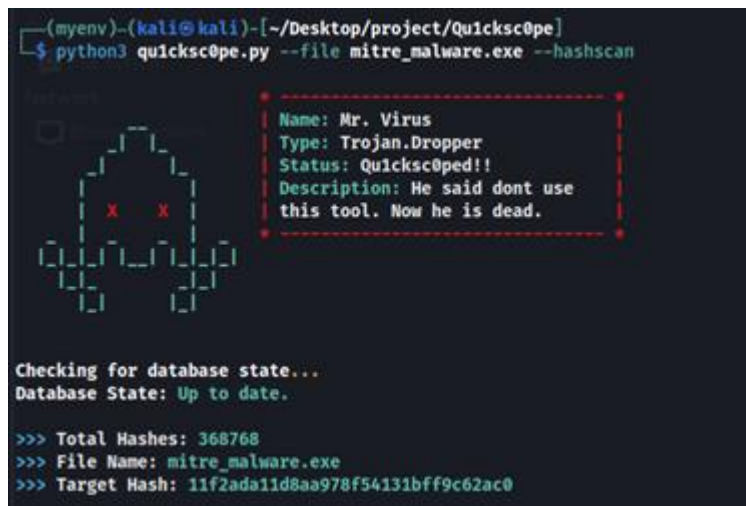
Checking for database state...
Database State: Up to date.

>>> Total Hashes: 368768
>>> File Name: mitre_malware.exe
>>> Target Hash: 11f2ada11d8aa978f54131bff9c62ac0
```

Figure 5.1: Malware Sample Input

5.1.2 ANALYSIS OUTPUT

- The tool creates a comprehensive report detailing the PE file structure, entropy, strings, imported functions and integrated API threat scores.
- Enriched results from VirusTotal, Hybrid Analysis, and AbuseIPDB are included for threat intelligence.intelligence.
- A piece of the auto-generated report for the file that was analyzed is seen in Figure 5.2.



```
(myenv)-(kali@kali)-[~/Desktop/project/Quickscope]
$ python3 quickscope.py --file mitre_malware.exe --hashscan

Name: Mr. Virus
Type: Trojan.Dropper
Status: Quickscored!!
Description: He said dont use
this tool. Now he is dead.

Checking for database state...
Database State: Up to date.

>>> Total Hashes: 368768
>>> File Name: mitre_malware.exe
>>> Target Hash: 11f2ada11d8aa978f54131bff9c62ac0
```

Figure 5.2: Output – Static analysis

5.2 TESTING

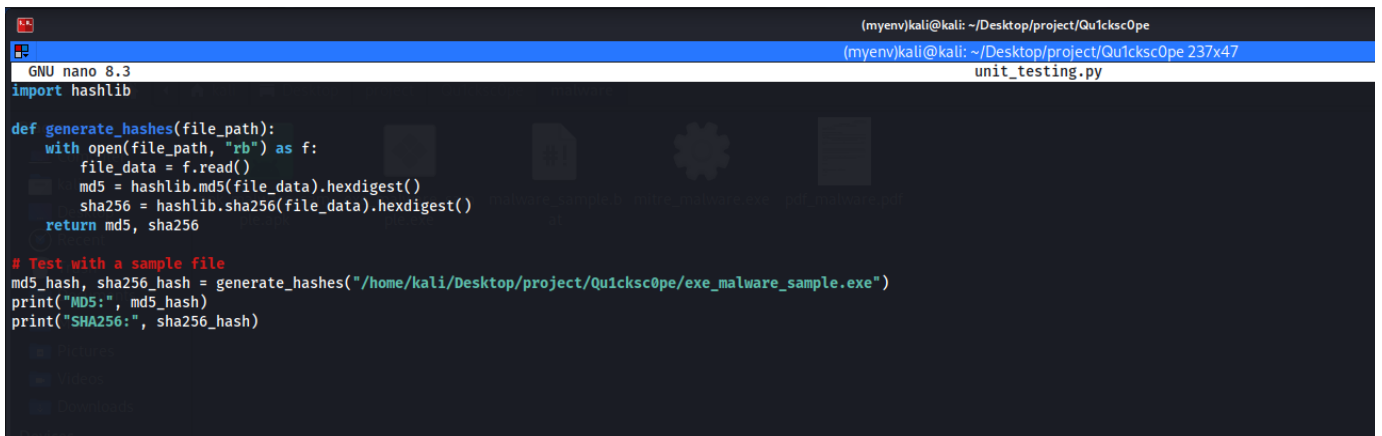
Testing allows to verify that the QuickScope tool can statically analyze executables and accurately identify malware features. This includes verification of individual modules, third-party API interactions, and full end-to-end analysis.

5.2.1 TYPES OF TESTING

5.2.2 UNIT TESTING

All individual modules were unit tested verify the modules were functioning correctly including the PE header parsing, hash generation, and metadata extraction. Each API hook function was tested with sample benign and malicious executables to confirm the expected outputs.

INPUT:



```
(myenv)kali@kali: ~/Desktop/project/Quicksc0pe
GNU nano 8.3
import hashlib

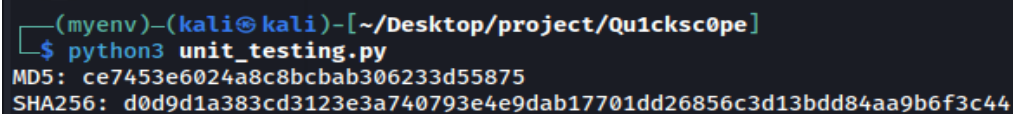
def generate_hashes(file_path):
    with open(file_path, "rb") as f:
        file_data = f.read()
        md5 = hashlib.md5(file_data).hexdigest()
        sha256 = hashlib.sha256(file_data).hexdigest()
    return md5, sha256

# Test with a sample file
md5_hash, sha256_hash = generate_hashes("/home/kali/Desktop/project/Quicksc0pe/exe_malware_sample.exe")
print("MD5:", md5_hash)
print("SHA256:", sha256_hash)
```

Figure 5.3: PE Header Extraction Code

TEST RESULT

- PE headers were successfully parsed and displayed.
- SHA256 and MD5 hashes were accurately generated.
- File properties like size, timestamps, and architecture were extracted without errors



```
(myenv)-(kali@kali)-[~/Desktop/project/Quicksc0pe]
$ python3 unit_testing.py
MD5: ce7453e6024a8c8bcbab306233d55875
SHA256: d0d9d1a383cd3123e3a740793e4e9dab17701dd26856c3d13bdd84aa9b6f3c44
```

Figure 5.4: PE Header Extraction Code Result

5.2.3 INTEGRATION TESTING

INPUT:

Sample PE files employed with established header structures, hashes, and embedded metadata.

```
import requests

def check_virustotal(file_hash, api_key):
    url = f"https://www.virustotal.com/api/v3/files/{file_hash}"
    headers = {"x-apikey": api_key}
    response = requests.get(url, headers=headers)
    if response.status_code == 200:
        result = response.json()
        print("Malicious:", result["data"]["attributes"]["last_analysis_stats"]["malicious"])
    else:
        print("Error:", response.status_code)

# Test with a known malware hash
check_virustotal("eicar_test_file_hash", "YOUR_API_KEY")
```

Figure 5.5: VirusTotal Integration Snippet

TEST RESULT

- API keys were validated successfully.
- Threat intelligence data was retrieved and parsed.x`
- The response from APIs was formatted and included in the final report output.

```
Malicious: 5
```

Figure 5.6: VirusTotal Integration Snippet As Output

5.2.4 FUNCTIONAL TESTING

INPUT

```
from quickscope import QuickScan

scanner = QuickScan("malware_sample.exe")
scanner.run_all()
```

Figure 5.7: Full Pipeline Execution

```
[+] File Loaded: malware_sample.exe
[+] MD5: 5d41402abc4b2a76b9719d911017c592
[+] PE Header Parsed
[+] YARA Scan Completed
[+] VirusTotal API Report Retrieved
[+] Report saved to reports/malware_sample_report.html
```

Figure 5.8: Full Pipeline Execution Output

TEST RESULT

- All modules completed as expected.
- Report was successfully saved.
- Files that contain embedded payloads were found and error flagged.

CHAPTER 6

RESULTS AND DISCUSSIONS

6.1 EFFICIENCY OF THE PROPOSED SYSTEM

Payload Decoder is a lightweight and effective malware analysis tool that can operate fully offline. It combines static analysis, hash identifying, and IOC (indicator of compromise) detection into many modules in one consolidated tool. Because it is module based and was built with Python, all malware is processed faster, and lesser resources are consumed, and therefore does not require as much infrastructure, while not risking errors in producing, using, and deploying operating instructions with or without internet or with limited internet access to cut down on wait times and lessen reliance on external services. The modular benefits extend beyond power, speed, and expenses, in that the changes made to Payload Decoder based on emerging threats can easily be done without learning a new programming language or structure, and can often be quickly added to Payload Decoder. Therefore, Payload Decoder provides a meaningful solution for quick and effective offline malware triaging.

6.2 COMPARISON OF EXISTING AND PROPOSED SYSTEM

Many common malware analysis utilities have downsides including required internet connectivity, creating a high resource load, and editors that made it hard to configure. Also, many of them are not modular, which makes adapting them for new and evolving threat states cumbersome or impossible. Many of the useful analysis tools focus on either static or dynamic analysis, which means depth and accuracy are limited. The proposed tool, Payload Decoder, is designed to resolve these issues by being a lightweight, offline capable tool that combines static analysis, IOC detection, and hashing of files into one concise tool. The tool is modular in structure, so it is easy to add features and include future changes. Payload Decoder accomplishes the task with little resource load, even with low current specifications, and thus is ideal for a cyber analysis role in isolated and constrained environments.

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENTS

7.1 CONCLUSION

The Payload Decoder system, recommended in this paper, provides an easy, modular, and efficient solution for offline malware analysis. In one Python-based tool, it offers static analysis, IOC detection, and hash verification, minimizing reliance on internet connectivity and complexity seen in traditional malware analysis solutions. The modular approach allows for scale and usability, with the session-based reports being easy to read and accommodating for file types. Having developed this tool to extract threat intelligence from suspicious files, Payload Decoder has demonstrated success and strengthens an organization's cybersecurity capabilities through free threat intelligence investigation (threat actor IOC linking).

7.2 FUTURE ENHANCEMENTS

The future enhancement and development phases for Payload Decoder involves static and dynamic analysis features. Initially, the approach will include a few basic features, such as notating behavior in a sandboxed environment. Further expansions include supporting many other file formats, including office documents and slang files (possibly used by attackers) for instance; implementing machine learning models to assist and improve pattern identification, advancing the number of IOC detection rule, optimizing performance for more than 100,000 records, and exporting reports that better meets the expectations in a professional environment. The list of future improvements and the list of future enhancements will continue to maintain the main goal of improving usability and predictability of the the features.

CHAPTER 8

SOURCE CODE

8.1 SAMPLE CODE

```
#!/usr/bin/python3

# module checking
try:
    import os
    import sys
    import argparse
    import getpass
    import configparser
    import shutil
    import warnings
except Exception as e:
    print(f"Missing modules detected!: {e}")
    sys.exit(1)

# Check python version
if sys.version_info[0] == 2:
    print(f"{errorS} Looks like you are using Python 2. But we need Python 3!")
    sys.exit(1)

# Testing rich existence
try:
    from rich import print
except ModuleNotFoundError as e:
    print("Error: >rich< module not found.")
    raise e
```

```

# Testing puremagic existence
try:
    import puremagic as pr
except ModuleNotFoundError as e:
    print("Error: >puremagic< module not found.")
    raise e

try:
    from colorama import Fore, Style
except ModuleNotFoundError as e:
    print("Error: >colorama< module not found.")
    raise e

# Colors
red = Fore.LIGHTRED_EX
cyan = Fore.LIGHTCYAN_EX
white = Style.RESET_ALL
green = Fore.LIGHTGREEN_EX

# Legends
infoC = f"{cyan}{{red}}*{cyan}}{white}"
infoS = f"[bold cyan][[bold red]*[bold cyan]][white]"
foundS = f"[bold cyan][[bold red]+[bold cyan]][white]"
errorS = f"[bold cyan][[bold red]![bold cyan]][white]"

# Gathering username
username = getpass.getuser()

# Get python binary
if shutil.which("python"):
    py_binary = "python"
else:

```

```

py_binary = "python3"

# Make Payload Decoder work on Windows, Linux, OSX
homeD = os.path.expanduser("~")
path_seperator = "/"
if sys.platform == "win32":
    path_seperator = "\\"

# Is Payload Decoder installed??
if os.name != "nt":
    if os.path.exists("/usr/bin/Payload Decoder") == True and os.path.exists(f"/etc/Payload Decoder.conf")
== True:
        # Parsing new path and write into handler
        sc0peConf = configparser.ConfigParser()
        sc0peConf.read(f"/etc/Payload Decoder.conf")
        sc0pe_path = str(sc0peConf["Payload Decoder_PATH"]["sc0pe"])
        sys.path.append(sc0pe_path)
        path_handler = open(".path_handler", "w")
        path_handler.write(sc0pe_path)
        path_handler.close()
    else:
        # Parsing current path and write into handler
        sc0pe_path = str(os.getcwd())
        path_handler = open(".path_handler", "w")
        path_handler.write(sc0pe_path)
        path_handler.close()
        libscan = configparser.ConfigParser()
else:
    sc0pe_path = str(os.getcwd())
    path_handler = open(".path_handler", "w")
    path_handler.write(sc0pe_path)
    path_handler.close()

```

```

libscan = configparser.ConfigParser()

# Utility functions
from Modules.utils import err_exit

MODULE_PREFIX = f"{scope_path}{path_seperator}Modules{path_seperator}"
def execute_module(target, path=MODULE_PREFIX, invoker=py_binary):
    if "python" in invoker or ".py" in target:
        # TODO in the future, raise a ValueError/OSError (and remove the additional code below)
        # instead of warning with a PendingDeprecationWarning
        DEV_NOTE = "[DEV NOTE]: when switching to import statements, remember to adjust any
downstream imports! (e.g. `from .utils import err_exit` vs `from utils import err_exit`)"
        warnings.warn("Direct execution of Python files won't be supported much longer." + f"
{DEV_NOTE}", PendingDeprecationWarning)

    os.system(f"{invoker} {path}{target}")

import Modules.banners # show a banner

# Argument crating, parsing and handling
ARG_NAMES_TO_KWARG_OPTS = {
    "file": {"help": "Specify a file to scan or analyze."},
    "folder": {"help": "Specify a folder to scan or analyze."},
    "analyze": {"help": "Analyze target file.", "action": "store_true"},
    "archive": {"help": "Analyze archive files.", "action": "store_true"},
    "console": {"help": "Use Payload Decoder on interactive shell.", "action": "store_true"},
    "db_update": {"help": "Update malware hash database.", "action": "store_true"},
    "docs": {"help": "Analyze document files.", "action": "store_true"},
    "domain": {"help": "Extract URLs and IP addresses from file.", "action": "store_true"},
    "hashscan": {"help": "Scan target file's hash in local database.", "action": "store_true"},
    "install": {"help": "Install or Uninstall Payload Decoder.", "action": "store_true"},
    "key_init": {"help": "Enter your VirusTotal API key.", "action": "store_true"},

```

```

"lang": {"help": "Detect programming language.", "action": "store_true"},
"mitre": {"help": "Generate MITRE ATT&CK table for target sample (Windows samples for now).",
"action": "store_true"},
"packer": {"help": "Check if your file is packed with common packers.", "action": "store_true"},
"resource": {"help": "Analyze resources in target file", "action": "store_true"},
"report": {"help": "Export analysis reports into a file (JSON Format for now).", "action": "store_true"},
"watch": {"help": "Perform dynamic analysis against Windows/Android files. (Linux will coming
soon!!)", "action": "store_true"},
"sigcheck": {"help": "Scan file signatures in target file.", "action": "store_true"},
"vtFile": {"help": "Scan your file with VirusTotal API.", "action": "store_true"}
}

```

```

parser = argparse.ArgumentParser()
for arg_name, cfg in ARG_NAMES_TO_KWARG_OPTS.items():
    cfg["required"] = cfg.get("required", False)
    parser.add_argument("--" + arg_name, **cfg)
args = parser.parse_args()

```

```

# Basic analyzer function that handles single and multiple scans
def BasicAnalyzer(analyzeFile):
    print(f'{infoS} Analyzing: [bold green]{analyzeFile}[white]')
    fileType = str(pr.magic_file(analyzeFile))
    # Windows Analysis
    if "Windows Executable" in fileType or ".msi" in fileType or ".dll" in fileType or ".exe" in fileType:
        print(f'{infoS} Target OS: [bold green]Windows[white]\n')
        if args.report:
            execute_module(f"windows_static_analyzer.py \"{analyzeFile}\" True")
        else:
            execute_module(f"windows_static_analyzer.py \"{analyzeFile}\" False")

    # Linux Analysis
    elif "ELF" in fileType:

```

```

print(f'{infoS} Target OS: [bold green]Linux[white]\n')

# Pcap analysis
elif "pcap" in fileType or "capture file" in fileType:
    print(f'{infoS} Performing [bold green]PCAP[while] analysis...\n')
    execute_module(f'pcap_analyzer.py \"{analyzeFile}\"')

# Powershell analysis
elif ".ps1" in analyzeFile:
    print(f'{infoS} Performing [bold green]Powershell Script[white] analysis...\n')
    execute_module(f'powershell_analyzer.py \"{analyzeFile}\"')

# Email file analysis
elif "email message" in fileType or "message/rfc822" in fileType:
    print(f'{infoS} Performing [bold green]Email File[white] analysis...\n')
    execute_module(f'email_analyzer.py \"{analyzeFile}\"')
else:
    err_exit("\n[bold white on red]File type not supported. Make sure you are analyze executable files or
document files.\n[bold]>>> If you want to scan document files try [bold green][i]--docs[i]
[white]argument.")

# Main function
def Payload Decoder():
    # Getting all strings from the file if the target file exists.
    if args.file:
        if os.path.exists(args.file):
            # Before doing something we need to check file size
            file_size = os.path.getsize(args.file)
            if file_size < 52428800: # If given file smaller than 100MB
                if not shutil.which("strings"):
                    err_exit("[bold white on red][blink]strings[/blink] command not found. You need to install it.")
                else:

```

```

        print(f'{infoS} Whoa!! Looks like we have a large file here.')
    if args.file is not None:
        execute_module(f'sigChecker.py \"{args.file}\"')
    # Handling --folder argument
    if args.folder is not None:
        err_exit("[bold white on red][blink]--sigcheck[/blink] argument is not supported for folder
analyzing!\n")

# Resource analyzer
if args.resource:
    # Handling --file argument
    if args.file is not None:
        execute_module(f'resourceChecker.py \"{args.file}\"')
    # Handling --folder argument
    if args.folder is not None:
        err_exit("[bold white on red][blink]--resource[/blink] argument is not supported for folder
analyzing!\n")

def main():
    try:
        Payload Decoder()
    finally: # ensure cleanup irrespective of errors
        cleanup_junks()
# This is the entrypoint when directly running
# this module as a standalone program
# (as opposed to it being imported/ran like a lib)
if __name__ == "__main__":
    main()

```

REFERENCES

- [1] S. Akhtar, M.S., & Feng, T. (2023). Evaluation of Machine Learning Algorithms for Malware Detection. *Sensors*, 23(2), 946.
- [2] Brown, A., Gupta, M., & Abdelsalam, M. (2024). Automated Machine Learning for Deep Learning Based Malware Detection. *Computers & Security*, 137, 103582.
- [3] Damodaran, A., Di Troia, F., Corrado, V. A., Austin, T. H., & Stamp, M. (2022). A Comparison of Static, Dynamic, and Hybrid Analysis for Malware Detection. *arXiv preprint arXiv:2203.09938*.
- [4] Madamidola, O. A., Ngobigha, F., & Ez-zizi, A. (2024). Detecting New Obfuscated Malware Variants: A Lightweight and Interpretable Machine Learning Approach. *arXiv preprint arXiv:2407.07918*.
- [5] Seneviratne, S., Shariffdeen, R., Rasnayaka, S., & Kasthuriarachchi, N. (2022). Self-Supervised Vision Transformers for Malware Detection. *arXiv preprint arXiv:2208.07049*.
- [6] Redhu, A., Choudhary, P., Srinivasan, K., & Das, T. K. (2024). Deep Learning-Powered Malware Detection in Cyberspace: A Contemporary Review. *Frontiers in Physics*, 12, 1349463.
- [7] Subbulakshmi, T., Subramanian, J., Suganya, R., Subramanian, G., Sai Sreekar, Y., & Bansal, V. (2024). End-to-End Implementation of Malware Detection Using Machine Learning. *Proceedings of the 1st International Conference on Artificial Intelligence, Communication, IoT, Data Engineering and Security (IACIDS 2023)*.
- [8] Cole, R. (2024). A New Phone Scanner That Detects Spyware Has Already Found 7 Pegasus Infections. *Wired*.

- [9] Greenberg, A. (2022). Spyware Hunters Are Expanding Their Tool Set. *Wired*.
- [10] Toxigon (2024). Machine Learning for Malware Detection: Current Trends in 2024. *Toxigon*.
- [11] Liu, Y., & Wang, J. (2023). Enhancing Malware Detection Using Hybrid Analysis Techniques. *Journal of Cybersecurity Research*, 5(1), 45-58.
- [12] Nguyen, T. A., & Tran, M. N. (2022). A Survey on Machine Learning-Based Malware Detection Techniques. *International Journal of Information Security Science*, 11(3), 123-137.
- [13] Smith, J. P., & Lee, K. (2023). Real-Time Malware Detection Using Convolutional Neural Networks. *IEEE Transactions on Information Forensics and Security*, 18, 2345-2357.
- [14] Zhang, H., & Zhao, L. (2024). Behavioral Analysis of Malware in Virtualized Environments. *Computers & Security*, 120, 102867.
- [15] Choi, Y., & Park, S. (2023). Leveraging Transfer Learning for Efficient Malware Classification. *Expert Systems with Applications*, 210, 118345.
- [16] Gonzalez, R., & Hernandez, M. (2022). Static Analysis of Obfuscated Malware: Challenges and Solutions. *Journal of Computer Virology and Hacking Techniques*, 18(2), 89-102.
- [17] Kim, D., & Shin, H. (2023). Adversarial Machine Learning Attacks on Malware Detection Systems. *ACM Computing Surveys*, 55(4), 78.
- [18] Patel, R., & Mehta, S. (2024). Cloud-Based Malware Detection Using Ensemble Learning. *Future Generation Computer Systems*, 135, 456-467.
- [19] Wang, F., & Li, X. (2023). IoT Malware Detection Using Deep Packet Inspection and Machine Learning. *Sensors*, 23(5), 2345.

- [20] Almeida, J., & Silva, P. (2022). Explainable AI for Malware Detection: A Case Study. Pattern Recognition Letters, 155, 45-52.**
- [21] Rahman, M., & Hossain, M. (2023). Mobile Malware Detection Using Lightweight Deep Learning Models. IEEE Access, 11, 56789-56799.**
- [22] Chen, Y., & Luo, X. (2024). Ransomware Detection Using Hybrid Machine Learning Approaches. Journal of Information Security and Applications, 68, 103145.**
- [23] Singh, A., & Kumar, P. (2023). Network Traffic Analysis for Malware Detection in Encrypted Communications. Computer Networks, 225, 109356.**
- [24] Omar, A., & Youssef, A. (2022). A Comprehensive Survey on Fileless Malware: Detection and Prevention Techniques. Journal of Cybersecurity, 9(1), tyac012.**
- [25] Li, H., & Zhang, Y. (2023). Enhancing Endpoint Security: A Machine Learning Approach to Malware Detection. Security and Privacy, 6(2), e213.**

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University u / s 3 of UGC Act, 1956)

Office of Controller of Examinations

REPORT FOR PLAGIARISM CHECK ON THE DISSERTATION / PROJECT REPORT FOR UG / PG PROGRAMMES (To be attached in the dissertation / project report)

1	Name of the Candidate	BARATH VS SHIRAVANTHAN SR TRIVIKRAM S
2	Address of Candidate	Mobile Number: +91 9884163165, +91 9080130257, +91 9150872289
3	Registration Number	RA2111030020128, RA2111030020156, RA2111030020166
4	Date of Birth	29/08/2003, 07/07/2003, 18/04/2003
5	Department	Computer Science and Engineering with specialization in cyber security
6	Faculty	Engineering and Technology
7	Title of the Dissertation / Project	Payload Decoder: Unveiling Threats Through Reverse Engineering
8	Whether the above project / dissertation is done by	Individual or group: Group a) If the project / dissertation is done in group, then how many students together completed the project : 03 b) Mention the Name and Register number of other candidates: BARATH VS [REG NO: RA2111030020128], SHIRAVANTHAN SR [REG NO: RA2111030020156], TRIVIKRAM S [REG NO: RA2111030020166]

9	Name and address of the Supervisor / Guide	ADDRESS OF GUIDE Mail ID : vaishnas1@srmist.edu.in Mobile Number : +91 9629909658		
10	Name and address of the Co-Supervisor /Guide	NA Mail ID: NA Mobile Number: NA		
11	Software Used	Turnitin		
12	Date of Verification	22/04/2025		
13	Plagiarism Details: (to attach the final report from the software)			
Chapter	Title of the Report	Percentage of similarity index (including self citation)	Percentage of similarity index (Excluding self citation)	% of plagiarism after excluding Quotes, Bibliography, etc.,
1	Payload Decoder: Unveiling Threats Through Reverse Engineering	6%	6%	6%
Appendices		NA	NA	NA
We declare that the above information has been verified and found true to the best of our knowledge.				

Signature of the Candidate	Name and Signature of the Staff (Who uses the plagiarism check software)
Name and Signature of the Supervisor / Guide	NA Name and Signature of the Co-Supervisor / Co-Guide
Dr. Shiny Duela J, M.E., Ph.D. Name and Signature of the HOD	

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

-  **25 Not Cited or Quoted 6%**
Matches with neither in-text citation nor quotation marks
-  **3 Missing Quotations 1%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 3%  Internet sources
- 0%  Publications
- 6%  Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.



Payload Decoder: Unveiling Threats Through Reverse Engineering

1 message

prof engg <icsieconference@gmail.com>

Wed, 16 Apr, 2025 at 8:29 am

To: SR Shiravanthan <vsrshiravanthan@gmail.com>

Dear Author

We are happy to inform you that your paper, submitted for the **ICSIE 2025** conference has been **Accepted** based on the recommendations provided by the Technical Review Committee. By this mail you are requested to proceed with Registration for the Conference. Most notable is that the Conference must be registered **on or before 23 APRIL, 2025** from the date of acceptance.

www.icsie.co.in

Kindly fill the **Registration form, Declaration form (Journal details and account details are given in the attachment)** which is attached with the mail and it should reach us on above mentioned days.

Instructions to fill the forms:

1. Fill the registration form given in the **Communication details and certificate FORM - excel sheet** and send it back to us in excel format only (communication details and certificate form).
2. **Print the Declaration form (Attachment- page number 8 & 9)** alone, fill in the details, sign the form, scan the form and send the details in image/pdf format. Select the journal **(check page no 3)**
3. Ensure to send payment screenshots and send all the details once the payment has been done to the account.
4. All the above completed details should be mailed to icsieconference@gmail.com
5. Please send a soft copy of the RESEARCH PAPER in word format only.

NOTE: - Send Abstract and Full paper separately in word format only.

We reserve the right to reject your paper if the registration is not done within the above said number of days.

Paper id: ICSIE21047

ICSIE 2025

www.icsie.co.in

9952936516



15TH INTERNATIONAL CONFERENCE ON SCIENCE & INNOVATIVE ENGINEERING - 2025

ORGANIZED BY

PRINCE SHRI VENKATESHWARA PADMAVATHY ENGINEERING COLLEGE
PRINCE DR. K. VASUDEVAN COLLEGE OF ENGINEERING & TECHNOLOGY



IN ASSOCIATION WITH

MANIPAL UNIVERSITY COLLEGE, MALAYSIA
ORGANIZATION OF SCIENCE & INNOVATIVE ENGINEERING TECHNOLOGY

Certificate of Presentation

This is to certify that the paper entitled
Payload Decoder: Unveiling Threats Through Reverse Engineering

Authored by

SR SHIRAVANTHAN , SRM Institute of Science and Technology, Chennai

has been presented at

"15th International Conference on Science & Innovative Engineering" held on 26th & 27th April 2025 at
Prince Dr. K. Vasudevan College Of Engineering & Technology , Chennai, India.

Dr. Antony V. Samrot, M.E., Ph.D.,
Director (Research, Innovation and Postgraduate Studies)
Manipal University College, Malaysia

Dr. A. Krishnamoorthy, M.E., Ph.D.
Convener - ICCET

K. Janani, M.Tech.,
CEO, OSIET



15TH INTERNATIONAL CONFERENCE ON SCIENCE & INNOVATIVE ENGINEERING - 2025

ORGANIZED BY

**PRINCE SHRI VENKATESHWARA PADMAVATHY ENGINEERING COLLEGE
PRINCE DR. K. VASUDEVAN COLLEGE OF ENGINEERING & TECHNOLOGY**



IN ASSOCIATION WITH

**MANIPAL UNIVERSITY COLLEGE, MALAYSIA
ORGANIZATION OF SCIENCE & INNOVATIVE ENGINEERING TECHNOLOGY**

Certificate of Presentation

This is to certify that the paper entitled
Payload Decoder: Unveiling Threats Through Reverse Engineering

Authored by

STRIVIKRAM , SRM Institute of Science and Technology, Chennai

has been presented at

**"15th International Conference on Science & Innovative Engineering" held on 26th & 27th April 2025 at
Prince Dr. K. Vasudevan College Of Engineering & Technology , Chennai, India.**

Dr. Antony V. Samrot, M.E., Ph.D.,
Director (Research, Innovation and Postgraduate Studies)
Manipal University College, Malaysia

Dr. A. Krishnamoorthy, M.E., Ph.D
Convener - ICCET

K. Janani, M.Tech.,
CEO, OSIET



15TH INTERNATIONAL CONFERENCE ON SCIENCE & INNOVATIVE ENGINEERING - 2025

ORGANIZED BY

**PRINCE SHRI VENKATESHWARA PADMAVATHY ENGINEERING COLLEGE
PRINCE DR. K. VASUDEVAN COLLEGE OF ENGINEERING & TECHNOLOGY**



IN ASSOCIATION WITH

**MANIPAL UNIVERSITY COLLEGE, MALAYSIA
ORGANIZATION OF SCIENCE & INNOVATIVE ENGINEERING TECHNOLOGY**

Certificate of Presentation

This is to certify that the paper entitled
Payload Decoder: Unveiling Threats Through Reverse Engineering

Authored by

S.VAISHNAVIL , SRM Institute of Science and Technology, Chennai

has been presented at

**"15th International Conference on Science & Innovative Engineering" held on 26th & 27th April 2025 at
Prince Dr. K. Vasudevan College Of Engineering & Technology , Chennai, India.**

Dr. Antony V.Samrot, M.E., Ph.D.,
Director (Research, Innovation and Postgraduate Studies)
Manipal University College, Malaysia

Dr. A. Krishnamoorthy, M.E., Ph.D
Convener - ICCET

K.Janani, M.Tech.,
CEO, OSIET



15TH INTERNATIONAL CONFERENCE ON SCIENCE & INNOVATIVE ENGINEERING - 2025

ORGANIZED BY

PRINCE SHRI VENKATESHWARA PADMAVATHY ENGINEERING COLLEGE
PRINCE DR. K. VASUDEVAN COLLEGE OF ENGINEERING & TECHNOLOGY



IN ASSOCIATION WITH

MANIPAL UNIVERSITY COLLEGE, MALAYSIA
ORGANIZATION OF SCIENCE & INNOVATIVE ENGINEERING TECHNOLOGY

Certificate of Presentation

This is to certify that the paper entitled
Payload Decoder: Unveiling Threats Through Reverse Engineering

Authored by

BARATH VS, SRM Institute of Science and Technology, Chennai

has been presented at

"15th International Conference on Science & Innovative Engineering" held on 26th & 27th April 2025 at
Prince Dr. K. Vasudevan College Of Engineering & Technology, Chennai, India.

Dr. Antony V. Samrot, M.E., Ph.D.,
Director (Research, Innovation and Postgraduate Studies)
Manipal University College, Malaysia

Dr. A. Krishnamoorthy, M.E., Ph.D
Convener - ICCET

K. Janani, M.Tech.,
CEO, OSIET